*I wasn't really sure what to explain about the app itself, so I just focused on the **API** part using **Dio** and explained **Bloc** and **Cubit**.*

*I also practiced making a simple **Counter App** with a theme switcher (Light/Dark mode). Right now, I'm working on the **Characters Viewer** app (from Omar Ahmed's tutorial) to practice my **UI** again since it's been a while, and to follow along with a professional workflow.*

Here is the repo link for the Characters app:
https://github.com/mahmoudAbdelghani2/characters_viewr

I'm still working on it, so it might be finished by the time you check it. I didn't upload the Counter App because it's very basic. Also, since I'm still working on the UI of the Characters app, I haven't written much about it yet.

That's pretty much it!

Subject | API = Application Programming Interface | Date

← هو حلقة الوصل بين 2 أبلكيشن عشان يتبادلوا البيانات ويكلموا بعض
من غير ما كل واحد يحتاج يعرف التاني متبرمج إزاي

* زي ما يكون عندك أبلكيشن بقلاتر للطقس معندوش Database داخلية فيها
درجات حرارة كل مكان ← هنا بنستخدم API وتربطه بالأبلكيشن

* الـ API بيحدد لك إزاي تسأل السيرفر (إيه الرابط / الـ EndPoint / إزاي الداتا الراجعة)
وبيحدد برضو السيرفر هيرد عليك بأي شكل

* مصطلحات ومكونات الـ API الأساسية

1 Base URL (a) & EndPoint (b)

(a) دة الرابط الأساسي وعنوان السيرفر نفسه مش بيتغير

(b) دي نقطة النهاية ودة المسار اللي بيحدد إنت عاوز إيه بالظبط من السيرفر
لو عاوز طقس /weather ، لو عاوز بيانات المستخدم /user

full Link → https://api.weatherapp.com/weather

2 الـ Methods → HTTP (إزاي السيرفر تتعامل إيه؟ تضيف - تمسح - تقرأ)

GET ← بتجيب بيانات من السيرفر ← السيرفر بيرد بداتا فقط

POST ← بتبعت إنت بيانات للسيرفر عشان يحفظها (تبعت باسورد وإيميل مثلاً)

PUT/PATCH ← بتعدل في بيانات موجودة أصلاً (زي تغير صورة البروفايل مثلاً)

DELETE ← بتمسح بيانات من السيرفر (زي إنك تلغي أورد)

3 Status Codes ← السيرفر بيرد عليك بأكواد

الـ 200 ← نجاح / 404 ← الداتا مش موجودة ، 400 ← إنت بعت داتا بشكل غلط

401 ← Unauthorized (غير مسموح ليك) / 500 ← غلطات من السيرفر

4 الـ Headers ، Body

Header ← دي معلومات بتبعتها مع الـ request مش بتظهر للمستخدم (Header زي الـ Token)

Body ← دي الداتا الفعلية اللي بتبعتها في حالات الـ POST ، PUT (JSON Code)

إنما الـ GET غالباً ملوش Body

Mohamed & Amira Notebooks

* مفاهيم مهمة في ال API

← ال JSON ← JavaScript Object Notation بيكون شكل زي ال Map ويكون هو لغة التواصل

← Serialization / Encoding ← عملية إرسال داتا للسيرفر بتحول الداتا لـ JSON

← DeSerialization / Decoding لما السيرفر يرد عليك بـ JSON بتاخده تحوله لـ object أو Model عشان يتعرض في ال UI بسهولة

← Query Parameters

هنا إنك تجيب بيانات محددة من بيانات ضخمة بتضيف parameter في آخر ال End Point بتاعت ال GET Request وبتتفصل بعلامة استفهام

[https:// ~~~~~~ /weather ? q = zagazig & units = metric]

← Pagination (تقسيم البيانات لصفحات)

هنا لو بتبعت request وفيه الآلاف البيانات السيرفر يرد عليك بـ 20 بس ولما تعمل scroll بيبعت request تاني يجيب ضربهم ودة بيتم عن طريق ال query parameters [~~~ ? page = 1 & limit = 20]

← Authentication & Authorization

API key ← زي باسور بيتبعت مع ال request عشان السيرفر يعرف إنك مسجل عنده قبل كدة وتطبيقك له صلاحية

Token ← لما اليوزر بيعمل login السيرفر بيديله Token مخصوص له عشان كل request هنعمله بعد كدة محتاج تبعت ال Token دة في ال header عشان يعرف إنه اليوزر مسجل دخول ولا لا وإيه صلاحياته

← Rate limiting ← السيرفر بيحدد عدد معين للـ Requests بتاعتك اللي تقدر تبعتها في وقت معين لو زدت عن كدة يرد عليك بـ Status code غالباً (429)

---

* مكتبة ال Dio ← موجودة عشان تسهل عمليات ال API / أحسن من HTTP عشان بتوفر علينا كتابة كود عدد سطوره كبيرة

الملفات

* Lib → networking → api_service.dart ← Dio init

* Lib → models → news_model.dart ← JSON ⇄ object

* Lib → ~~Networking~~ → Web_services → news_api.dart ← Requests

① api_service.dart

```
Class ApiService {
    Late Dio dio;
    ApiService() {
    Base options options= Base options(
        baseURL: 'https://newsapi.org/v2/',
        receiveTimout: Duration(seconds: 10),
        ConnectTimeout: Duration(seconds: 10),
    );
    dio = Dio(options)
    }
```

* ممكن هنا نبعت ال Token

```
headers: {
'Authorization': 'Token',
```

* ممكن نبعت ال queryParameters الثابتة مش هتتغير في ال request

```
queryParameters: {
'Language': 'ar',
},
```

* فيه validateStatus عشان لو جه اي status code غير ال 200 ال dio هترمي exception

② Models / news_model.dart

هنا بقى الاحسن آخد عنصر واحد من ال List اللي جواها Map اللي هو ال Json ابعته لاي موقع converter Json to Dart عشان يحولي كل الاتريبيوتس وانا ابعت احدها بنفسي

Simple ex:-

```
Class ArticleModel {
    Final String title;
    Final String desc;
  ArticleModel ({ required this. title, ... });
    Factory ArticleModel. From JSON (Map<String, dynamic> json) {
        return ArticleModel (
            title : json['title'] ?? "بدون عنوان",
            desc : json['desc'] ?? "بدون وصف",
        );
    }

    Map<String, dynamic> to Json () {
        return {
            'title': title,
            'desc': desc,
        };
    }
```

ده استقبال البيانات من ال API

ممكن تعملها لو حابب تحفظ داتا local & وتبعت إنت للسيرفر بيانات من عندك

Mohamed & Amira NOTEBOOKS

③ Web_services / news_api.dart

```
Class NewsApi {
    Final ApiService apiservice;
    NewsApi (this.apiservice);
    Future<List<ArticleModel>> getTopNews() async {
      try {
        Response response = await apiservice.dio.get(
            'top-headLineas',
            queryParameters : {
              'Country' : 'eg',
              'api key' : 'your api key',
            },
        );
      List<dynamic> articles Json = response.data['articles'];
      List<ArticleModel> articles = articlesJson.map(
          (article) => ArticleModel.fromJson(article)).toList();

      return articles;
      } Catch(e) {
        print('Error : $e');
        return [];
      }
    }
}
```

لسة فيه برضو غير ال dio.get
dio.Post(), dio.Put()
dio.delete()
وكل واحدة لها خصائصها
ودة لو ال api بيسمحلك تغير
في الداتا حاجة ، لو مسمحش بتستخدم ال dio.get() بس

Subject | State management Date

⇐ كيفية التحكم في البيانات المتغيرة أثناء تشغيل التطبيق ومتابعة التغيرات عشان يتم تحديث الـ UI على أساسه

* BloC

↳ قبل الـ BloC كنا بنكتب الكود كله في مكان واحد داخل في بعض (API Calls + UI + Business Logic) في نفس الـ screen
ده كان بيسبب مشاكل كتير

* Spaghetti Code ← كل الكود داخل في بعض
* مشكلة الـ Rebuild ← إنك تعمل setState عشان تغير حاجة بسيطة فا تبني الشاشة كلها ده شيء سيء

↳ الـ BloC يعتبر Architecture بيفصل كل حاجة عن التانية تماماً عشان المشروع يكون Scalable - Clean

* Bloc Architecture

[1] Presentation layer [2] Business logic layer [3] Data layer

(1) UI [Buttons - Text - Widgets] أي حاجة اليوزر بيشوفها

* الوظيفة هي إنها تبعت (Events) لما اليوزر يعمل action ويستقبل (States) عشان تحدث شكل الشاشة

(2) BloC or Cubit → Business logic

* وظيفته هي إنه بيستقبل الـ Events من الـ UI ويروح للـ Data layer يطلب منه الداتا المناسبة ولما الداتا توصله على أساسها بيبعت State ويرجعها للـ UI
ex: loading State - Loaded State

③ Data Layer → مسئولة عن جلب البيانات سواء من API أو Db

① Web Services (API)

وظيفته إنه يكلم الـ API ويجيب الـ JSON زي ما هو ← Dio - http

② Models

الـ Classes اللي فيها بحول الـ JSON لـ Data object وظيفة نقدر نتعامل معاهم بسهولة

③ Repository

دة اللي بيكلم Web services وبعدين يجيب الداتا يبعتها لـ Models وبعدين ياخد الداتا من الـ Model يبعتها لـ Bloc أو Cubit

* Data flow

UI (event) → Bloc → Repository → Web services → JSON → Repository → Model → Repository → Bloc → UI

loading state

State

Json

* Bloc vs. Cubit

* الـ Cubit هو نسخة مصغرة من الـ Bloc يعتمد على الـ functions لتغيير الحالة ← الـ UI بتستدعي function مباشرة من داخل الـ Cubit والـ function بتعمل حاجة معينة وتطلق حالة جديدة باستخدام emit

⇐ ex ← عند الضغط على Button لتحديث بيستدعي Cubit.fetch() فالـ Cubit بعمل emit(Data load)

* الـ Bloc هو النسخة الأكبر والأكثر تقدماً يعتمد على الـ Events بدل من الـ function

← UI بيبعت Event لـ Bloc والـ Bloc يعمل اللي مفروض يعمله ثم يطلق حالة جديدة (emit)

الـ Cubit مناسب للأبلكيشنز الصغيرة والمتوسطة وكود أقل

الـ Bloc مناسب لكل حاجة بس أحسن للأبلكيشنز الكبيرة لأنه كود أكبر

Mohamed & Amira Notebooks

* 4 Widgets to Connect UI ⇄ Bloc Cubit

① BlocProvider

هو اللي بيعمل Creat لل Bloc أو ال Cubit وبيوفرهم للـ Widgets اللي تحته في ال Widget tree

← بيشتغل بنظام ال Lazy loading مش هيعمل Create إلا لما ال UI يطلب الأول عشان يوفر في ال memory

② BlocBuilder

هو اللي بيبني ال UI أو بيعمل rebuild في كل مرة ال State تتغير فيها

خلي ال Bloc Builder أحسن في الأماكن اللي بيتعمله rebuild فقط مش فوق ال Scaffold عشان يحسن ال Performance

③ BlocListener

هو اللي بيـ Listen على ال State عشان ينفذ Action لمرة واحدة فقط

هو مش بيعمل rebuild لل UI خالص بتستخدمه مع الحاجات اللي محتاجة تصميم يتغير زي SnackBar - Toast - وقت Dialog أو BottomSheet على Navigation لشاشة تانية

④ BlocConsumer

عبارة عن ميكس بين الاثنين (Listener & Builder) في Widget واحدة

بتستخدمه لو إنت عندك شاشة محتاج لها الداتا تحمل تعرضها في List ومن نفس الوقت لو فيه error يطلع Dialog